KTH Royal Institute of Technology

SF2527 Numerical Methods for Differential Equations I

Computer Exercise 1

Ordinary Differential Equations and Elliptic Equations

Exercise number: Computer Exercise 1
Group: ____________________

Student	Name	Personal number	Email
1	____________________	_______________	____________________
2	____________________	_______________	____________________

HT26

August 29, 2026

Contents

1 Runge–Kutta accuracy and stability

The method used throughout Parts 1 and 2 is

$$k_1 = f(t_n, u_n), \qquad k_2 = f(t_n + h, u_n + hk_1), \tag{1.1}$$

$$k_3 = f\left(t_n + \frac{h}{2}, u_n + \frac{h}{4}k_1 + \frac{h}{4}k_2\right), \qquad u_{n+1} = u_n + \frac{h}{6}(k_1 + k_2 + 4k_3). \tag{1.2}$$

It is an explicit three-stage, one-step Runge–Kutta method.

1.1 Landau–Lifshitz system

Let

$$a = \frac{1}{4}\begin{bmatrix} 1 \\ \sqrt{11} \\ 2 \end{bmatrix}, \qquad C = \frac{1}{4}\begin{bmatrix} 0 & -2 & \sqrt{11} \\ 2 & 0 & -1 \\ -\sqrt{11} & 1 & 0 \end{bmatrix}.$$

Since $Cm = a \times m$, the ODE is

$$m' = Am, \qquad A = C + \alpha C^2, \qquad \alpha = 0.07, \qquad m(0) = \begin{bmatrix} 0 & 0 & 1 \end{bmatrix}^\mathsf{T}.$$

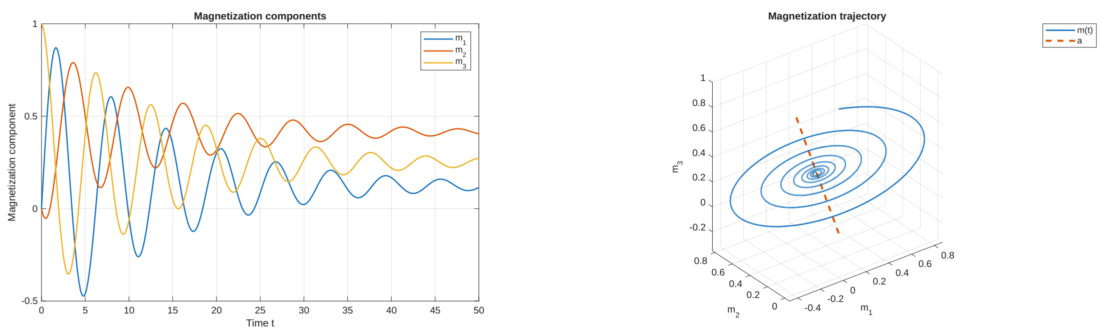

Figure 1: Magnetization components and trajectory for $0 \le t \le 50$ with $h = 0.01$.

The scalar product $a^\mathsf{T} m(t) = a^\mathsf{T} m(0) = 1/2$ is constant. Thus the trajectory lies in an affine plane normal to a. The perpendicular component is damped and

$$m(t) \longrightarrow (a^\mathsf{T} m(0))a = \frac{1}{2}a,$$

which agrees with the plots.

1.2 Order of accuracy

For $N = 50, 100, 200, 400, 800, 1600$, let $h = 50/N$ and

$$d_N = \|\widetilde{m}_N(50) - \widetilde{m}_{2N}(50)\|_2 .$$

The fitted log–log slope is

$$\boxed{p = 2.994893},$$

confirming third-order global accuracy.

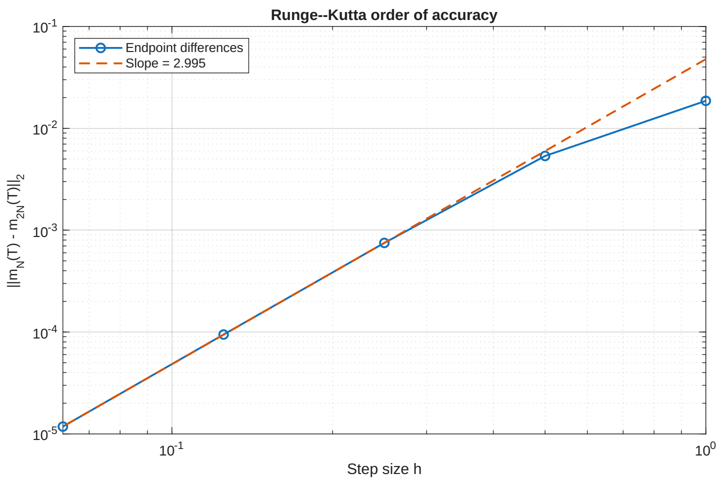

Figure 2: Endpoint differences and fitted order.

1.3 Stability limit

The eigenvalues of A are

$$\lambda_1 = 0, \qquad \lambda_{2,3} = -0.07 \pm i.$$

The stability function is

$$R(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6}.$$

Stability requires $|R(h\lambda_j)| \leq 1$ for every eigenvalue. The zero eigenvalue gives $R(0) = 1$. For $\lambda = -0.07 + i$, the first positive root of

$$|R(h\lambda)| - 1 = 0$$

was computed with `fzero` on $[2, 2.5]$, giving

$$\boxed{h_0 = 2.055596}.$$

The run with $h = 2$ is stable, while $h = 50/24 = 2.083333$ is unstable.

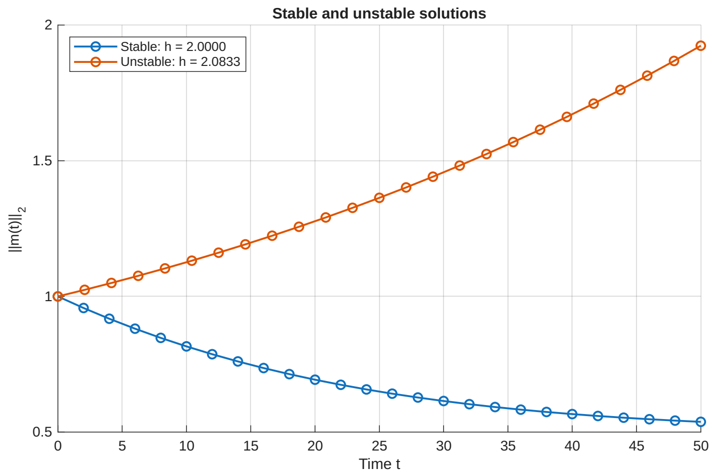

Figure 3: Stable and unstable solutions on opposite sides of h_0.

2 Robertson's stiff system

The concentrations $x = [x_A, x_B, x_C]^\mathsf{T}$ satisfy

$$x_A' = -r_1 x_A + r_2 x_B x_C, \tag{2.1}$$
$$x_B' = r_1 x_A - r_2 x_B x_C - r_3 x_B^2, \tag{2.2}$$
$$x_C' = r_3 x_B^2, \tag{2.3}$$

where

$$r_1 = 5.0 \times 10^{-2}, \qquad r_2 = 1.2 \times 10^4, \qquad r_3 = 4.0 \times 10^7, qquad x(0) = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}^{\mathsf{T}}.$$

The invariant $x_A + x_B + x_C = 1$ was preserved numerically.

2.1 Explicit Runge–Kutta method

On $0 \le t \le 10$, $h = 7.0 \times 10^{-4}$ gave a stable solution. The linear plot hides the small intermediate concentration x_B; the logarithmic plot shows its fast initial transient.

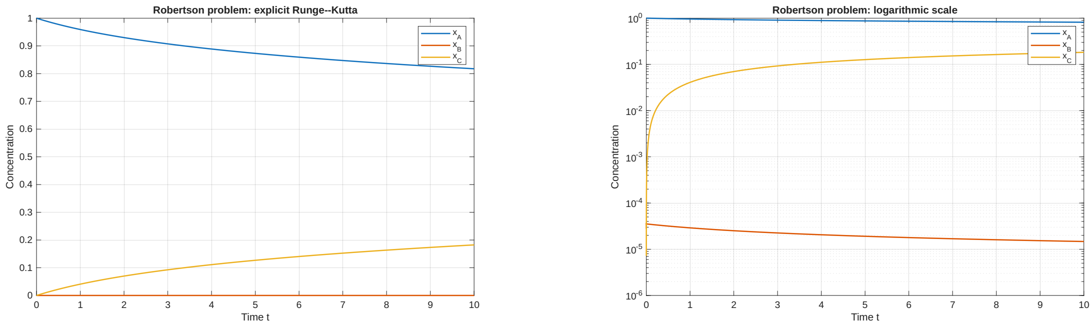

Figure 4: Explicit RK solution on $0 \le t \le 10$ with $h \approx 7.0 \times 10^{-4}$.

The Jacobian is

$$J(x) = \begin{bmatrix} -r_1 & r_2 x_C & r_2 x_B \\ r_1 & -r_2 x_C - 2r_3 x_B & -r_2 x_B \\ 0 & 2r_3 x_B & 0 \end{bmatrix}.$$

For a perturbation e, $e' \approx J(x(t))e$, so local stability requires $|R(h\lambda_j(t))| \le 1$. One eigenvalue is zero. Along the computed solution,

$$\max_t |\lambda_j(t)| = 3365.924700.$$

The negative real-axis boundary of the RK region gives

$$h_{\max} \approx 7.4652 \times 10^{-4},$$

in agreement with the empirical value. The restriction becomes more severe as the large eigenvalue grows in magnitude.

For the long integration, only the current state was stored.

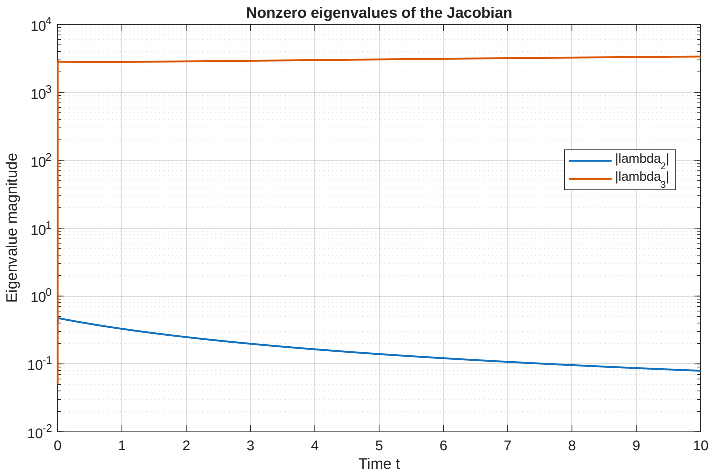

Figure 5: Magnitudes of the two nonzero Jacobian eigenvalues.

Table 1: Explicit RK result at $t = 1000$.

Quantity	Value
h	2.8×10^{-4}
$x_A(1000)$	0.293414227164
$x_B(1000)$	$1.716342048384 \times 10^{-6}$
$x_C(1000)$	0.706584056494
Time	2.93 s

2.2 Implicit Euler

Implicit Euler satisfies

$$u_{n+1} = u_n + hf(u_{n+1}).$$

At each step, the supplied code solves

$$F(v) = v - hf(v) - u_n = 0$$

with Newton's method:

$$(I - hJ_f(v))d = -F(v), \qquad v \leftarrow v + d.$$

Runs with $h = 10$, 1 and 0.1 remained bounded. The large steps are stable but inaccurate. The choice $h = 0.1$ visually agrees with RK on $[0, 10]$ and was reused on $[0, 1000]$.

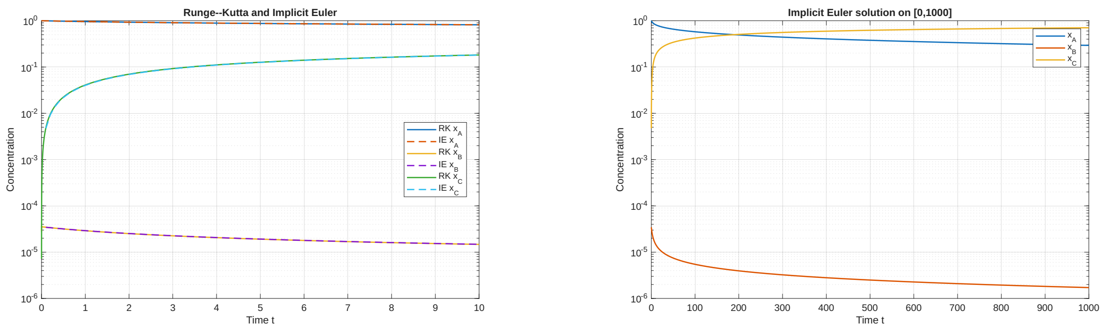

Figure 6: Short-time RK/IE comparison and the IE solution on $[0, 1000]$.

2.3 Efficiency

Errors were measured against the supplied reference value at $t = 1000$. The IE timing version stores only the current state.

IE is faster for moderate accuracy because stability permits large steps. For high accuracy, its first-order error and Newton solves dominate; RK is faster below an error of roughly 10^{-6}. For the nonstiff problem in Part 1, IE has no useful stability advantage, so the higher-order explicit method is preferable.

Table 2: Endpoint error and single-run computational time.

Method	h	Error	Time (s)
RK	2.8×10^{-4}	3.06×10^{-13}	2.93
IE	1	4.93×10^{-4}	0.014
IE	0.1	5.03×10^{-5}	0.048
IE	0.01	5.04×10^{-6}	0.409
IE	0.001	5.04×10^{-7}	4.04

3 One-dimensional convection–diffusion equation

The problem is

$$-T''(z) + vT'(z) = Q(z), \qquad 0 < z < 1,$$

with $T(0) = T_0$ and

$$-T'(1) = \alpha(v)(T(1) - T_{\text{out}}), \qquad \alpha(v) = \sqrt{\frac{v^2}{4} + \alpha_0^2} - \frac{v}{2}.$$

The parameters are

$$a = 0.1,\ b = 0.4,\ Q_0 = 7000,\ \alpha_0 = 50,\ T_{\text{out}} = 25,\ T_0 = 100,$$

and

$$Q(z) = \begin{cases} Q_0 \sin\left(\dfrac{(z-a)\pi}{b-a}\right), & a \le z \le b, \\ 0, & \text{otherwise.} \end{cases}$$

Let $z_j = jh$, $h = 1/N$, and $U = [T_1, \dots, T_N]^{\mathsf{T}}$. At $j = 1, \dots, N-1$,

$$\ell T_{j-1} + dT_j + rT_{j+1} = Q(z_j),$$

where

$$\ell = -\frac{1}{h^2} - \frac{v}{2h}, \qquad d = \frac{2}{h^2}, \qquad r = -\frac{1}{h^2} + \frac{v}{2h}.$$

The known value T_0 is moved to the first right-hand-side entry. The second-order outlet row is

$$T_{N-2} - 4T_{N-1} + (3 + 2h\alpha)T_N = 2h\alpha T_{\text{out}}.$$

The resulting sparse linear system is solved with backslash.

3.1 Grid refinement

Table 3: Temperature at the exact grid point $z = 0.5$.

N	h	$T_h(0.5)$
80	0.0125	258.430491144194
160	0.00625	258.621578905525
320	0.003125	258.669336283685

Using successive differences gives

$$p = \log_2\left(\frac{|T_{80}(0.5) - T_{160}(0.5)|}{|T_{160}(0.5) - T_{320}(0.5)|}\right) = \boxed{2.000440}.$$

The source is evaluated at the same points z_j on every grid, and $z = 0.5$ is read at index $j = N/2$.

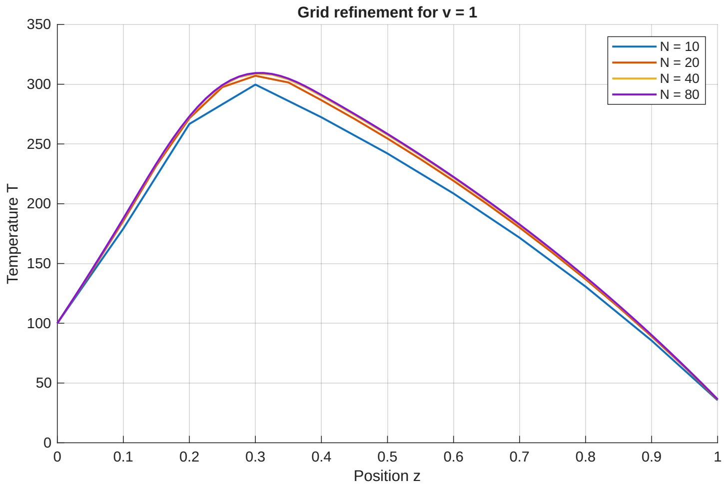

Figure 7: Solutions for $v = 1$ and $N = 10, 20, 40, 80$.

3.2 Velocity dependence

Table 4: Grids used for the velocity comparison.

v	N	h
1	80	0.0125
5	100	0.0100
15	300	0.003333
100	2000	0.0005

Larger v transports heat downstream more rapidly, so the temperature rise is smaller and the profile is flatter. It also produces a thinner outlet feature; the finer grids resolve this feature without oscillations.

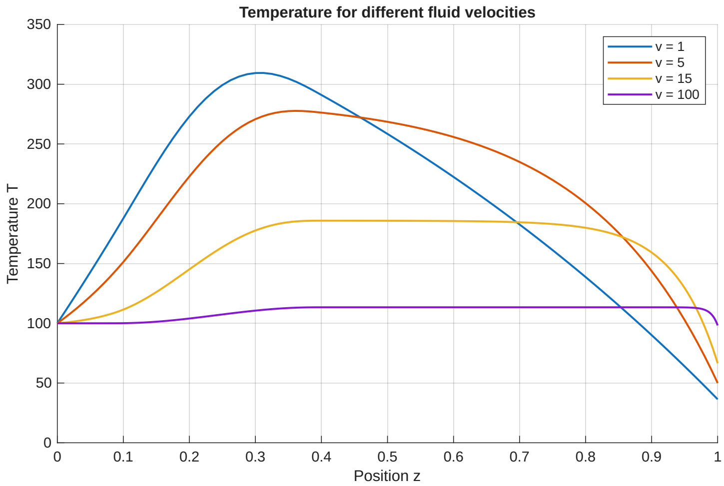

Figure 8: Temperature profiles for $v = 1, 5, 15, 100$.

4 Two-dimensional elliptic heat equation

The problem is

$$-\Delta T = f \quad \text{in } (0,12)\times(0,5),$$

with $T(x,0) = T_{\text{ext}} = 25$ and homogeneous Neumann conditions on the left, right and top boundaries. The mesh has

$$h = \frac{12}{N} = \frac{5}{M}.$$

The bottom values are known. All points with $i = 0,\dots,N$ and $j = 1,\dots,M$ are numbered by

$$k = i + 1 + (j-1)(N+1).$$

Thus the sparse matrix has size $(N+1)M \times (N+1)M$. At an interior point,

$$4T_{i,j} - T_{i-1,j} - T_{i+1,j} - T_{i,j-1} - T_{i,j+1} = h^2 f(x_i, y_j).$$

For $j = 1$, the known lower value adds T_{ext} to the right-hand side. The second-order boundary rows are

$$\begin{aligned} -3T_{0,j} + 4T_{1,j} - T_{2,j} &= 0,\\ 3T_{N,j} - 4T_{N-1,j} + T_{N-2,j} &= 0,\\ 3T_{i,M} - 4T_{i,M-1} + T_{i,M-2} &= 0. \end{aligned}$$

The upper corners use the side rows. This gives one equation for every unknown.

4.1 Constant source

For $f \equiv 2$ and $h = 0.2$, the computed value is

$$\boxed{T_h(6,2) = 41.000000000000}.$$

For an exact solution $T = c_0 + c_1 y + c_2 y^2$, the PDE and boundary conditions give

$$-2c_2 = 2, \qquad c_0 = 25, \qquad c_1 + 10c_2 = 0.$$

Hence

$$\boxed{T(x,y) = 25 + 10y - y^2}, \qquad T(6,2) = 41.$$

The central difference satisfies

$$\frac{g(x+h) - 2g(x) + g(x-h)}{h^2} = g''(x) + \frac{h^2}{12} g^{(4)}(x) + O(h^4).$$

For a quadratic, $g^{(3)} = g^{(4)} = 0$. Both the interior stencil and the one-sided boundary derivatives are therefore exact. The observed difference at $(6,2)$ was 3.70×10^{-13}, due only to floating-point arithmetic.

4.2 Localized source

For

$$f(x,y) = 100 \exp\left(-\frac{1}{2}(x-4)^2 - 4(y-1)^2\right),$$

the same matrix construction is used with a new right-hand side.

The successive-difference estimate is

$$\boxed{p = 2.059792},$$

which is consistent with second-order convergence. For the finest grid the system has 24100 unknowns, so sparse storage is essential.

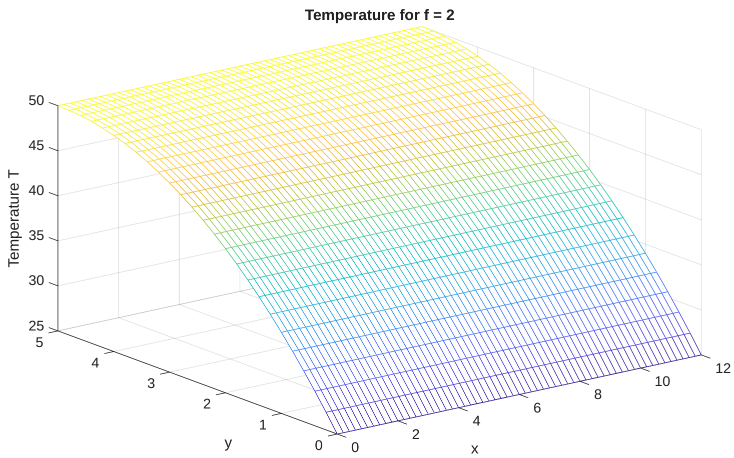

Figure 9: Numerical solution for $f \equiv 2$ and $h = 0.2$.

Table 5: Localized-source values at $(6, 2)$.

h	N	M	$T_h(6, 2)$
0.20	60	25	47.219954048124
0.10	120	50	47.224023288355
0.05	240	100	47.224999298104

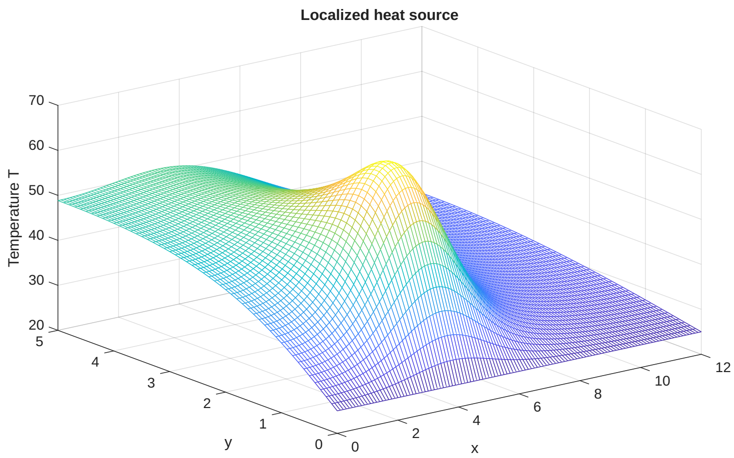

Figure 10: Mesh plot for the localized source with $h = 0.1$.

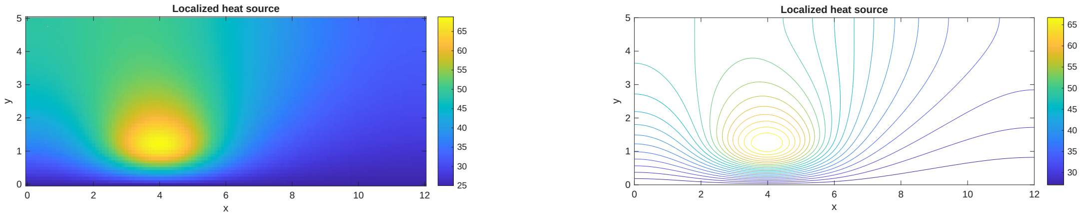

Figure 11: Image and contour views for $h = 0.1$. The hot region is centred near the source at $(4, 1)$.

5 Conclusions

The RK method showed third-order accuracy and the predicted stability limit. For the stiff Robertson problem, implicit Euler was efficient at moderate accuracy, while RK was preferable for high accuracy. Both finite-difference implementations showed second-order convergence. The analytic quadratic in Part 4 provided an exact verification of the two-dimensional matrix assembly.

Submitted MATLAB files

- `ce1_part1_rk_landau_lifshitz.m`, `rk3_step.m`
- `ce1_part2_robertson.m`, `impeuler.m`, `impeuler_nosave.m`
- `ce1_part3_convection_diffusion_1d.m`, `solve_convection_diffusion.m`
- `ce1_part4_elliptic_heat_2d.m`, `solve_heat_2d.m`